Siqiy - Vision, Concept & Feature Roadmap

> "I can start drawing immediately and add automation progressively, through exploration and iteration."

Working document - March 2026

1. Why this project?

Visio is the only tool that enables intelligent, scriptable diagrams, and it is fading. Wrong priorities at Microsoft, lack of innovation, and the rise of AI tools have created a critical gap.

Existing alternatives are either drawing tools without intelligence (Inkscape, draw.io), or purely code-first solutions that require a programmer (D3.js, Mermaid). The middle ground is empty.

Siqiy fills exactly that gap: an open, modern tool that is immediately usable and can gradually become as intelligent as needed.

2. The name: Siqiy

Siqiy (pronounced: *see-key*) is the Quechua word for "to draw a line" / "to draw".

Why Siqiy?

The name was selected from more than 30 candidates across multiple languages and checked against existing IT products. The decision criteria were:

- Sound: short, memorable, internationally pronounceable
- Distinctiveness: no collision with existing IT products, domains, or package names (npm, PyPI, GitHub)
- Meaning: direct etymological connection to drawing
- Availability: `siqiy.io` and related domains are unregistered

Eliminated candidates: VELA (Constellation Software), Diagen (University of Munich / drug discovery), Kartos (cybersecurity), Kroki (diagram API), Skema (BIM tool), Grafio (iOS app), Tecto (used in multiple contexts), Vizra (AI platform), SODA (SODA PDF), Gravo (Gravotech), and many more.

Siqiy stands on its own, without referencing Visio or imitating it. The next generation of users should not need to know Visio to understand Siqiy.

3. Core philosophy: Progressive Automation

The defining principle is **Progressive Automation**. No current tool offers this in this form:

1. I draw a shape and it works instantly, with no configuration.
2. I realize this shape should have behavior, so I open the ShapeSheet.
3. I add a formula and it takes effect live, immediately.
4. I realize multiple shapes need this, so I turn it into a master.
5. I realize this needs logic, so I add a script.

No break in flow. No mode switch into a separate programming environment. Users discover features while building, exactly like in Excel.

Nobody who discovers Siqiy needs prior Visio knowledge. The target group is the next generation of engineers, technicians, and designers who need complex, intelligent diagrams without having to program.

4. Positioning

- Drawing tools (Inkscape, Illustrator): intelligence no, accessibility yes

- Code-first tools (D3.js, Mermaid): intelligence yes, accessibility no
- Siqiy: intelligence yes, accessibility yes

Siqiy is not a competitor to Illustrator, Blender, or CAD. It is a platform for technical and structured diagramming: P&IDs, circuit diagrams, manufacturing drawings, infographics, business graphics.

Why previous OSS alternatives failed

draw.io is technically strong, but follows the wrong philosophy. It is a drawing program, not a framework. Shapes are not intelligent. There is no formula engine, no type system, no true scripting.

LibreOffice Draw is too bound to desktop paradigms, without web DNA or strong community momentum.

The real pattern of failure: they recreate the **appearance**, not the **intelligence**. Nobody seriously reimplemented the ShapeSheet concept. That is Siqiy's core.

5. The heart: ShapeSheet and formula engine

The ShapeSheet concept from Visio is adopted and modernized. Every shape is a living object with a table of formulas, like Excel cells that reference each other:

```
Width  = Height * 1.5
FillColor = IF(Prop.Status = "active", RGB(0,255,0), RGB(255,0,0))
LocPinX = Width * 0.5
```

This means shapes have behavior, not just appearance. Changes propagate automatically through a reactive dependency graph, like spatial Excel.

Improvements over Visio

- Expanded formula set: closes known Visio gaps
- Visual formula debugger: currently a black box in Visio
- Linkable properties: direct references between shape text and properties
- Unit-aware formulas: pressure in bar vs PSI, with automatic conversion

6. Master shapes, stencils, and dynamic references

Visio's model of drawings, templates, stencils, and scripts is preserved and extended with one key feature: **dynamic references**.

The problem in Visio

When a master from a stencil is inserted into a drawing, a local copy is created and the source link is lost. If the original shape changes, nothing updates.

The solution: reference vs copy

- Reference instance: live link to source shape, auto-updates
- Local copy: decoupled, independent, but can be rebound later
- Component overrides: local customization remains while structural source updates still apply

Each instance has an explicit binding status:

- `reference`: synchronized with source shape
- `copy`: decoupled, independent
- `outdated`: source changed, update pending
- `conflict`: local edits plus source changes, requires three-way merge

Company-wide stencils can evolve and all P&IDs worldwide can update automatically.

7. Type system for shapes

Visio has no formal type system. A master shape defines appearance and formulas, but not a class. Inheritance, interfaces, and polymorphism are missing.

Siqiy introduces a type system: shape types with inheritance, shared properties, and defined behavior. A P&ID shut-off valve is a subclass of valve, which in turn is a pipe component.

This enables standards validation, domain-specific queries, and consistent frameworks across drawings.

8. Scripting and automation

VBA declines together with Visio. The new scripting model is modern, open, and multilingual:

- TypeScript: primary language, framework builders work directly in TS
- Python: first-class via Pyodide (WebAssembly), with familiar ecosystem access

Event system

The event model should be at least as rich as Visio's:

- `Shape_Added`, `Shape_Deleted`, `Shape_Changed`
- `Cell_Changed` for formula updates
- `Connection_Added`, `Connection_Deleted`
- `Document_Saved`, `Page_Changed`
- Custom events definable by frameworks

9. Two-layer architecture

- Framework builders: experts (engineers, technicians, domain specialists) who define shape types, rules, formulas, and behavior
- End users: drafters, planners, designers who use finished frameworks without hidden complexity

The framework builder creates the P&ID tool. The carpenter uses it. Neither has to know the other's internals.

10. Modern features Visio never had

AI and MCP integration

- Shapes and properties are readable and writable by AI
- MCP server support: Claude, GPT, and others can interact with the document directly
- Semantic understanding: AI understands the diagram, not only the image
- Example: "Insert a DN50 valve between Pump 3 and Tank 2"

Git compatibility

- Text-based open format (JSON/XML) with meaningful diffs
- Merge conflicts in drawings can be resolved
- Framework and stencil versioning

Collaboration

- CRDT-based real-time editing (Yjs or Automerge)
- Offline-first with sync when available
- Conflict-free parallel editing

Time dimension

- Shape states: valve open / closed / fault
- State animation

- Simulation: a P&ID that can be played through
- Historical views: how the plant looked in 2019

Scale and units

- True physical scale, critical for carpenters, P&IDs, floor plans
- Shapes understand physical units: length, pressure, temperature
- Automatic conversion between unit systems

Drill-down and hierarchy

- A shape can contain a full diagram
- Clicking "Pump Group" opens the detailed drawing
- Bidirectional linking, similar to professional SCADA systems

Topology and network model

- Connectors understand what they connect, not only geometry
- Topology queries: "Show all shapes connected to Tank 3"
- Connectors with own properties and formulas

Smart connectors

- Orthogonal routing with obstacle avoidance
- Fixed anchor points (ports) with ShapeSheet behavior
- Live performant rerouting when shapes move
- Algorithm base: A-star on grids, extensible to visibility graphs

Validation engine

- Frameworks define rules and the tool checks them automatically
- Standards validation: validate a P&ID against ISO 10628
- Unit tests for shapes, testable like code

Live data integration

- Shapes display live sensor values from APIs or databases
- Bidirectional sync: diagram changes write back to data source
- OPC-UA for industrial plants

11. Plugin system

The plugin system is the multiplier. It is not just a feature, but a platform decision inspired by VS Code and Obsidian:

- Easy entry: a plugin can be built in a few hours
- Rich API: enough access for genuinely useful extensions
- Marketplace: discoverability, ratings, one-click install

Possible plugin categories:

- New shape libraries (P&ID, UML, network, electrical, ...)
- Export formats (DXF, IFC, STEP, SVG, PDF, ...)
- Data connectors (SQL, REST, GraphQL, OPC-UA, Excel, ...)
- Domain-specialized AI assistants
- Standards validators (ISO, DIN, ANSI, ...)
- Themes and UI extensions

12. Integration into existing ecosystems

Three levels

- Export: SVG, PDF, PNG, DXF (necessary, but lossy)
- Embedding: Office add-in, Confluence, Notion, Obsidian, VS Code while keeping diagrams alive
- Format as standard: open format others can implement, similar to Markdown

Browser renderer

A Web Component enables direct embedding into any website, turning diagrams into living elements in documentation.

Diagram as documentation

- Not export to Word: the diagram **is** the documentation
- Shapes carry descriptions, standards, warnings
- Publishable as interactive HTML
- Bills of materials and reports generated directly from shape properties

13. Technical stack

The stack was critically reviewed and consolidated in two rounds by seven independent LLM analyses. Initial proposals for hybrid rendering (PixiJS + SVG) and HyperFormula were unanimously rejected and replaced.

Consensus from external reviews

- PixiJS + SVG hybrid: rejected unanimously due to coordinate drift, duplicated hit-testing, and high maintenance cost
- HyperFormula: rejected unanimously because GPLv3 blocks OSS usage and grid semantics do not match shape graphs
- Yjs: clear consensus as industry standard with strong tooling
- React: clear consensus due to ecosystem advantage
- Tauri: clear consensus due to web-first model and low footprint compared to Electron

Rendering strategy by phase

The renderer is encapsulated behind a clean abstraction layer so it can be swapped without rewriting business logic.

Phase 1-2: Konva.js (Canvas 2D)

- Single rendering path without hybrid chaos
- Native sharp text without hacks
- Integrated object model aligns with ShapeSheet concept
- Scales to about 5000 shapes, enough for most diagrams

Phase 3+: PixiJS (pure WebGL if profiling requires it)

- Used only if performance measurements justify it
- WebGL only, no SVG overlay
- Text solved via SDF fonts

Architecture layers

```
+-------------------------------------------+
| UI Layer                                  |
| React + TypeScript + shadcn/ui            |
+-------------------------------------------+
| Renderer Interface (abstract)             |
```

```
| Konva.js Phase 1 -> PixiJS Phase 3          |
+---------------------------------------------+
| Reactive DAG (formula engine)               |
| Custom parser + Preact Signals              |
| Formula evaluation always local             |
+---------------------------------------------+
| CRDT Layer (Yjs)                            |
| Stores formulas, values, structure          |
| Does not evaluate                           |
+---------------------------------------------+
| Routing Engine                              |
| Phase 1: direct lines (manual)              |
| Phase 2: custom A* + ELK.js                 |
+---------------------------------------------+
| Scripting                                   |
| TypeScript native + Pyodide (plugin)        |
+---------------------------------------------+
| Desktop                                     |
| Tauri (Rust), web-first and desktop-ready   |
+---------------------------------------------+
```

Critical principle: formulas vs CRDT

This is the largest technical risk, explicitly highlighted by all seven reviews:

Formulas are evaluated locally and never stored as computed CRDT state.

Concretely:

- Yjs stores formula strings, raw property values, and document structure
- Local engine computes derived values from formulas
- On conflict (user A edits formula, user B deletes referenced shape), show `Broken Reference` marker instead of crashing; the formula remains and shows an error state

Strategy

Web-first, desktop-ready. Local data storage by default, no cloud dependency in core. Collaboration is architecturally prepared in phase 1, but optional. ShapeSheet has priority.

14. Community strategy

The strongest open-source projects emerge when the community feels ownership. Blender's turning point came with transparent governance. VS Code grew through its plugin ecosystem.

What attracts this community

- Open documented format so anyone can build tools around it
- Plugin system with low entry barrier
- Showcase gallery with complete frameworks that demonstrate possibilities
- Good documentation from day one
- Discord/forum with direct migration path for the visguy.com community

The strongest magnet

A well-known Visio expert publicly building their first framework in Siqiy. This is more authentic than marketing. Knowledge accumulated over years in the visguy.com forum becomes accessible instead of buried.

15. Inspirations from other projects

- Excel: no tutorial required, immediate start, then discover depth

- HyperCard (1987): direct parallel, cards with properties and scripts; users are also programmers
- Minecraft: no fixed goal, no tutorial, Redstone as progressive automation
- Grasshopper: parametric drawing for architects without programming
- Smalltalk: everything is an object that can be inspected and modified live
- Figma: component overrides, local changes plus global updates
- VS Code / Obsidian: plugin ecosystem as platform multiplier

16. High-level roadmap

- Phase 1 (MVP): shape with formula engine (dependency graph, reactive cells), master/stencil concept, file save/load, basic scripting API
- Phase 2 (Core): dynamic references (reference vs copy), shape type system, plugin system (API + marketplace), Python scripting via Pyodide
- Phase 3 (Platform): Git-compatible format, AI/MCP integration, CRDT collaboration, browser renderer / Office add-in
- Phase 4 (Ecosystem): validation engine, live data integration, drill-down hierarchy, time dimension / simulation

17. Formula syntax

Reference syntax is one of the truly irreversible decisions. It affects data model, serialization, Git diffs, plugin API, and AI readability.

Design principles

- Explicit over implicit: `Self.Width` instead of `Width`
- JavaScript-like: immediately understandable for JS and Python users
- AI-readable: self-explanatory without hidden context
- Name as alias, ID as anchor: users write names, system stores IDs, renames propagate like IDE refactoring

Casing convention

```
Self.Width          # system cell, uppercase means Siqiy-defined
Self.Height
Self.Prop.pressure  # custom property, lowercase means framework-defined
Self.Prop.material
Page.Width          # document-level, uppercase
```

Full syntax reference

```
# Geometry (system cells)
Self.Width    = Self.Height * 1.5
Self.LocPinX  = Self.Width * 0.5
Self.LocPinY  = Self.Height * 0.5
Self.Angle    = 0

# Custom properties
Self.Prop.pressure = 10
Self.Prop.color    = Self.Prop.pressure > 8 ? "#CC0000" : "#00AA00"

# Referencing other shapes (name -> internal ID)
Self.Width            = Shape("Tank1").Prop.volume / 100
Self.Prop.flow_rate   = Shape("Pump3").Prop.rated_power * 0.85

# Referencing connectors
Self.Prop.flow = Connector("Line1").Prop.flow_rate
```

```
# Page and document
Self.Width        = Page.Width * 0.1
Self.Prop.units   = Doc.Prop.unit_system

# Ports
Port("Inlet").X   = 0
Port("Inlet").Y   = Self.Height / 2
Port("Outlet").X  = Self.Width
Port("Outlet").Y  = Self.Height / 2

# Functions (lowercase, familiar)
Self.Width        = max(Self.Prop.min_width, Self.Height * 1.5)
Self.Prop.label   = concat(Self.Prop.type, "-", Self.Prop.number)
Self.FillColor    = rgb(255, 0, 0)

# Units (explicit, auto-convertible)
Self.Prop.pressure     = 10[bar]
Self.Prop.pressure_psi = Self.Prop.pressure[psi]

# Locking (modernized Visio GUARD)
Self.Width = locked(100)

# Error handling
Self.Prop.value = Shape("Tank1").Prop.volume ?? 0
```

Internal shape references

Users always write names. Siqiy stores IDs internally:

```
# User writes:
Self.Width = Shape("Pump3").Prop.rated_power * 0.85

# Siqiy stores internally:
Self.Width = Shape(#a3f9).Prop.rated_power * 0.85
```

If `Pump3` is renamed, Siqiy updates all formulas automatically. Users keep seeing names; the system uses stable IDs.

18. Data model

The data model is foundational for serialization, Git diffs, CRDT, plugin API, and AI readability.

Shape object

```
{
  "id": "a3f9",
  "name": "Pump3",
  "type": "ref:stencil/pid/pumps/centrifugal-pump",
  "master": {
    "ref": "stencil://pid-standard/centrifugal-pump",
    "binding": "reference",
    "overrides": ["cells.FillColor", "props.label"]
  },
  "cells": {
    "Width":     { "formula": "Self.Height * 1.5", "value": 60.0 },
    "Height":    { "formula": null, "value": 40.0 },
    "PinX":      { "formula": null, "value": 120.0 },
    "PinY":      { "formula": null, "value": 80.0 },
    "Angle":     { "formula": null, "value": 0.0 },
    "LocPinX":   { "formula": "Self.Width * 0.5", "value": 30.0 },
```

```
    "LocPinY":   { "formula": "Self.Height * 0.5", "value": 20.0 },
    "FillColor": { "formula": "Self.Prop.status == 'on' ? '#00AA00' : '#CC0000'",
"value": "#00AA00" },
    "LineColor": { "formula": null, "value": "#333333" },
    "Visible":   { "formula": null, "value": true },
    "Locked":    { "formula": null, "value": false }
  },
  "props": {
    "label": {
      "value": "P-101", "formula": null,
      "type": "string", "label": "Label", "unit": null
    },
    "status": {
      "value": "on", "formula": null,
      "type": "enum", "options": ["on", "off", "fault"],
      "label": "Operating state", "unit": null
    },
    "pressure": {
      "value": 8.5, "formula": null,
      "type": "number", "label": "Operating pressure", "unit": "bar"
    },
    "flow_rate": {
      "value": 120.0, "formula": "Shape(#b1e5).Prop.volume / 10",
      "type": "number", "label": "Flow rate", "unit": "m3/h"
    }
  },
  "ports": {
    "inlet": {
      "x": { "formula": "0", "value": 0.0 },
      "y": { "formula": "Self.Height / 2", "value": 20.0 },
      "direction": "left",
      "accepts": ["fluid-line"]
    },
    "outlet": {
      "x": { "formula": "Self.Width", "value": 60.0 },
      "y": { "formula": "Self.Height / 2", "value": 20.0 },
      "direction": "right",
      "accepts": ["fluid-line"]
    }
  },
  "children": [],
  "meta": {
    "created": "2026-03-08T10:00:00Z",
    "modified": "2026-03-08T14:23:00Z",
    "layer": "equipment",
    "tags": ["pump", "centrifugal-pump"],
    "notes": "Main supply pump cooling loop"
  }
}
```


Three design principles

Formula and value always together. Each cell carries both formula and last computed value. Rendering reads `value`, reactive DAG updates it as dependencies change. Git diffs show whether formula or value changed. AI can read documents without knowing internals of the formula engine.

Props are typed. Each custom property knows type, unit, and label. This enables auto-generated property panels, unit conversion, validation, and AI understanding without extra schema files.

Master binding lives in the shape. The `master` object stores origin, binding type, and local overrides. During stencil updates, Siqiy can precisely decide what to propagate and what to preserve.

Connector object

Connectors are first-class shapes with two special fields:

```
{
  "id": "c7d2",
  "name": "Line1",
  "type": "ref:stencil/pid/lines/fluid",
  "connection": {
    "from": { "shapeId": "a3f9", "port": "outlet" },
    "to":   { "shapeId": "b1e5", "port": "inlet" }
  },
  "cells": {
    "LineColor":  { "formula": null, "value": "#0055AA" },
    "LineWeight": { "formula": null, "value": 2.0 },
    "Routing":    { "formula": null, "value": "orthogonal" }
  },
  "props": {
    "medium":    { "value": "Water", "type": "string", "label": "Medium", "unit":
null },
    "flow_rate": { "value": 120.0, "type": "number", "label": "Flow rate", "unit":
"m3/h" }
  }
}
```

Document structure

```
{
  "siqiy": "1.0",
  "id": "doc-uuid",
  "meta": { "title": "Cooling Loop Plant A", "author": "...", "created": "..." },
  "units": { "length": "mm", "pressure": "bar", "temperature": "degC" },
  "pages": [
    {
      "id": "page-uuid",
      "name": "Overview",
      "cells": { "Width": 2970, "Height": 2100 },
      "shapes": []
    }
  ],
  "stencils": [],
  "scripts": []
}
```

Git diffs

A single value change yields a minimal readable diff:

```
- "value": "off",
+ "value": "on",
```

A formula change:

```
- "formula": "Self.Height * 1.5",
+ "formula": "Self.Height * 2.0",
```

```
- "value": 60.0,
+ "value": 80.0,
```

19. Stencil format

A stencil is a library of master shapes, the source of truth for all referencing instances.

Design goals

- Standalone file: readable and usable outside the tool
- Versionable: Git-friendly with traceable changes
- Distributable: URL, local file, or npm package
- Typed: defines both shapes and type hierarchy
- Documented: each shape carries its own documentation

Stencil file structure

```
pid-standard/
|- stencil.json
|- shapes/
|  |- centrifugal-pump.json
|  |- shutoff-valve.json
|  |- vessel.json
|  `- ...
|- geometry/
|  |- centrifugal-pump.svg
|  `- ...
|- scripts/
|  `- validation.ts
`- README.md
```

`stencil.json`

```
{
  "siqiy-stencil": "1.0",
  "id": "pid-standard",
  "name": "P&ID Standard (ISO 10628)",
  "version": "2.1.0",
  "author": "Siqiy Community",
  "license": "MIT",
  "description": "Standard stencil for piping and instrumentation diagrams according
to ISO 10628",
  "tags": ["P&ID", "Process Engineering", "ISO 10628"],
  "types": {
    "Component": {
      "description": "Base type for all physical components",
      "props": {
        "label":      { "type": "string", "label": "Label", "required": true },
        "manufacturer": { "type": "string", "label": "Manufacturer", "required":
false },
        "year_built": { "type": "number", "label": "Year built", "unit": "year" }
      }
    },
    "PipeComponent": {
      "extends": "Component",
      "description": "All components installed in piping",
      "props": {
        "nominal_size":  { "type": "number", "label": "Nominal size", "unit": "DN" },
        "nominal_pressure": { "type": "number", "label": "Nominal pressure", "unit":
```

```
"PN" },
        "material":      { "type": "string", "label": "Material" }
      }
    },
    "Pump": {
      "extends": "PipeComponent",
      "description": "Pumps of all types",
      "props": {
        "flow_rate":       { "type": "number", "label": "Flow rate", "unit": "m3/
h" },
        "head":            { "type": "number", "label": "Head", "unit": "m" },
        "drive_power":     { "type": "number", "label": "Drive power", "unit": "kW" }
      }
    },
    "CentrifugalPump": {
      "extends": "Pump",
      "description": "Centrifugal pumps (radial, axial, mixed flow)"
    }
  },
  "shapes": [
    "shapes/centrifugal-pump.json",
    "shapes/shutoff-valve.json",
    "shapes/vessel.json"
  ]
}
```

Master shape definition

```
{
  "id": "centrifugal-pump",
  "name": "Centrifugal Pump",
  "type": "CentrifugalPump",
  "description": "Centrifugal pump according to ISO 10628-2, symbol CP",
  "norm": "ISO 10628-2",
  "geometry": "geometry/centrifugal-pump.svg",
  "cells": {
    "Width":     { "formula": "Self.Height * 1.2", "default": 48.0 },
    "Height":    { "formula": null, "default": 40.0 },
    "LocPinX":   { "formula": "Self.Width * 0.5", "default": 24.0 },
    "LocPinY":   { "formula": "Self.Height * 0.5", "default": 20.0 },
    "FillColor": {
      "formula": "Self.Prop.status == 'fault' ? '#CC0000' : Self.Prop.status ==
'off' ? '#888888' : '#FFFFFF'",
      "default": "#FFFFFF"
    }
  },
  "props": {
    "status": {
      "type": "enum",
      "options": ["on", "off", "fault"],
      "default": "off",
      "label": "Operating state"
    }
  },
  "ports": {
    "suction": {
      "x": { "formula": "0" },
```

```
      "y": { "formula": "Self.Height / 2" },
      "direction": "left",
      "label": "Suction nozzle",
      "accepts": ["fluid-line", "gas-line"]
    },
    "discharge": {
      "x": { "formula": "Self.Width" },
      "y": { "formula": "Self.Height / 2" },
      "direction": "right",
      "label": "Discharge nozzle",
      "accepts": ["fluid-line", "gas-line"]
    }
  },
  "validation": [
    {
      "rule": "Self.Prop.nominal_size > 0",
      "severity": "warning",
      "message": "Nominal size should be provided"
    },
    {
      "rule": "Self.Prop.flow_rate <= 5000",
      "severity": "error",
      "message": "Flow rate exceeds allowed range"
    }
  ]
}
```

Binding model: reference vs copy

```
Stencil (source of truth)
 |
 +- Instance A [binding: reference]
 |  Inherits master, local overrides possible
 |  Stencil update -> instance updates
 |
 +- Instance B [binding: copy]
 |  Fully local copy, decoupled
 |  Stencil update -> no effect
 |
 `- Instance C [binding: reference, status: outdated]
    Master changed, update pending
    Siqiy warns, user decides
```

Stencil distribution

```
"stencils": [
  { "id": "pid-standard",   "source": "https://stencils.siqiy.io/pid-standard@2.1.
0" },
  { "id": "my-framework",   "source": "file://./stencils/my-framework" },
  { "id": "company-internal","source": "npm:@mycompany/siqiy-stencils@1.0.0" }
]
```

This enables centrally maintained company stencils: when a symbol changes, all referencing drawings can update worldwide.

20. ShapeSheet editor and error handling

The ShapeSheet editor is the most important UI element for framework builders. It is not a plain text field, but an intelligent formula IDE.

Smart cell editor

Internal syntax is precise and case-sensitive, but the editor tolerates input variation and normalizes it transparently:

```
# User types loosely (case-insensitive):
self.width = self.height * 1.5

# Editor normalizes on blur:
Self.Width = Self.Height * 1.5

# User types:
fillcolor = pump3.prop.status = "on" ? green : red

# Editor suggests:
FillColor = Shape("Pump3").Prop.status == "on" ? Green : Red
```

What the editor does:

- Auto-normalization of case when leaving a cell
- Fuzzy shape recognition (`pump3 -> Shape("Pump3")` when unambiguous)
- Autocomplete for shapes, properties, functions
- Live validation for syntax errors, missing references, type conflicts
- Formula explanation on hover in natural language
- Dependency visualization (incoming and outgoing)
- AI suggestions (for example: missing state-based FillColor)

Editing ergonomics has priority. Syntax is the internal format; the editor is the human interface.

Error model: `#ERR` instead of `#REF`

As in Visio, the last valid value remains active so the drawing keeps working. The editor shows errors clearly without blocking work.

```
{ "formula": "Self.Height * 1.5", "value": 60.0, "state": "ok" }

{ "formula": "Shape(#a3f9).Prop.width", "value": 60.0, "state": "#ERR:REF",
  "error": "Shape 'Pump3' not found" }

{ "formula": "Page.Width / Self.Prop.count", "value": 100.0, "state": "#ERR:DIV",
  "error": "Division by zero (count = 0)" }

{ "formula": "Self.Height", "value": 40.0, "state": "#ERR:CIRC",
  "error": "Circular reference: Width -> Height -> Width" }
```

Error codes:

- `#ERR:REF`: missing reference (shape, port, property)
- `#ERR:DIV`: division by zero
- `#ERR:TYPE`: type conflict in formula
- `#ERR:CIRC`: circular dependency in graph
- `#ERR:SYN`: formula syntax error
- `#ERR:UNIT`: incompatible units
- `#ERR:LOCK`: attempt to overwrite locked cell

The last valid `value` always remains. The drawing keeps functioning.

21. Delta storage

Principle

Most shapes in a drawing are master instances and differ from the source in only a few cells. Siqiy stores only this difference internally, not the full shape.

A P&ID with 200 similar valves does not store geometry 200 times. It stores only differences such as name, position, and a few properties.

Implementation: runtime vs disk

Delta storage is a transparent optimization. Users and APIs do not see complexity.

- Runtime memory: fully resolved objects
- Disk serialization: only deltas against master

```
{
  "id": "a3f9",
  "name": "Pump3",
  "master": "stencil://pid-standard/centrifugal-pump",
  "delta": {
    "cells": {
      "FillColor": { "formula": "Self.Prop.status == 'on' ? Green : Red", "value":
"#00AA00" }
    },
    "props": {
      "label": { "value": "P-101" },
      "pressure": { "value": 8.5 }
    }
  }
}
```

```
{
  "id": "a3f9",
  "cells": {
    "Width": { "formula": "Self.Height * 1.2", "value": 48.0 },
    "Height": { "formula": null, "value": 40.0 },
    "FillColor": { "formula": "Self.Prop.status == 'on' ? Green : Red", "value":
"#00AA00" }
  }
}
```

On load, Siqiy resolves deltas automatically. On save, Siqiy computes deltas automatically. Formula engine and renderer always operate on full objects.

Collaboration and deltas

Yjs always synchronizes fully resolved objects, not deltas. Delta computation is strictly a serialization step. This avoids conflict-resolution complexity.

Missing master fallback

```
{
  "state": "#ERR:REF",
  "error": "Master 'stencil://pid-standard/centrifugal-pump' not found",
  "fallback": "last-known-full"
}
```

Siqiy stores a snapshot of the last successfully resolved full object. If the master is missing, drawing continues with this snapshot and a visible warning.

22. Style system

Styles are CSS-like. They define reusable appearance profiles that are applied automatically by rules. The core principle is cascade: more specific rules override more general ones; multiple rules can contribute to different properties.

Four cascade levels

```
Level 1: Base style        (document default, like CSS reset)
Level 2: Type style        (defined on master shape)
Level 3: Rule style        (data-driven, automatic)
Level 4: Direct assignment (manual, highest priority)
```

Example: flowsheet styles

A valve's appearance can be fully derived from properties without manual styling:

```
{
  "styles": {
    "medium.product": { "cells": { "FillColor": "#CC0000" } },
    "medium.water":   { "cells": { "FillColor": "#0055CC" } },
    "medium.steam":   { "cells": { "FillColor": "#FF8800" } },
    "medium.cip":     { "cells": { "FillColor": "#00AA55" } },

    "priority.sample": {
      "cells": { "LineWeight": 0.5, "CornerRadius": 0, "TextVisible": false }
    },
    "priority.main": {
      "cells": { "LineWeight": 1.5 }
    },

    "geometry.1D": { "cells": { "CornerRadius": 0 } },
    "geometry.2D": { "cells": { "CornerRadius": 3 } }
  },

  "style-rules": [
    { "condition": "Self.Prop.medium == 'Product'", "apply": "medium.product" },
    { "condition": "Self.Prop.medium == 'Water'",   "apply": "medium.water" },
    { "condition": "Self.Prop.medium == 'Steam'",   "apply": "medium.steam" },
    { "condition": "Self.Prop.medium == 'CIP'",     "apply": "medium.cip" },
    { "condition": "Self.Prop.priority == 'Sample'", "apply": "priority.sample" },
    { "condition": "Self.Prop.geometry == '1D'",     "apply": "geometry.1D" },
    { "condition": "Self.Prop.geometry == '2D'",     "apply": "geometry.2D" }
  ]
}
```

Result for `Valve.123` (`medium=Product`, `priority=Sample`, `geometry=1D`):

- `medium.product` -> `FillColor = #CC0000`
- `priority.sample` -> `LineWeight = 0.5`, `CornerRadius = 0`, `TextVisible = false`
- `geometry.1D` -> `CornerRadius = 0` (redundant but harmless)

If two rules target the same property, the more specific one wins, or the later one in list order (framework-configurable).

Styles and views

A view can selectively enable or disable style rules. For example, a print plan can disable color rules and apply a black-and-white style.

Styles in the data model

Styles primarily live in the stencil because they are part of the framework. Drawings may define local styles, but framework styles come from stencils and propagate through updates.

23. Layer system

Layers combine AutoCAD precision with modern tag-like semantics. Shapes can belong to multiple layers simultaneously. Layers are classifications, not drawers.

Hierarchical layers with custom properties

```
{
  "layers": {
    "architecture": {
      "label": "Architecture",
      "color": "#333333",
      "props": { "trade": "Construction" },
      "children": {
        "walls":   { "label": "Walls" },
        "doors":   { "label": "Doors & Windows" },
        "ceilings":{ "label": "Ceilings" }
      }
    },
    "electrical": {
      "label": "Electrical",
      "color": "#FFAA00",
      "props": { "trade": "Electrical", "contractor": "Electrical Ltd." },
      "children": {
        "power": {
          "label": "Power",
          "children": {
            "outlets":  { "label": "Outlets" },
            "panels":   { "label": "Panels" }
          }
        },
        "low_voltage": {
          "label": "Low Voltage",
          "children": {
            "data_cable": { "label": "Data Cable" },
            "telephone":  { "label": "Telephone" }
          }
        }
      }
    },
    "furniture": {
      "label": "Furniture",
      "color": "#00AA55",
      "children": {
        "office":     { "label": "Office" },
        "meeting":    { "label": "Meeting" }
      }
    }
  }
}
```

Shapes in multiple layers

```
{
  "id": "a3f9",
  "name": "Desk-42",
  "layers": ["furniture.office", "electrical.low_voltage.data_cable"]
}
```

This desk appears in the furniture plan **and** the data-cable plan.

Views: saved visual configurations

Views combine layer visibility, opacity, color overrides, and print settings into reusable named setups:

```
{
  "views": {
    "electrical-plan": {
      "label": "Electrical Plan",
      "layers": {
        "architecture":          { "visible": true,  "opacity": 0.3, "locked":
true },
        "electrical":            { "visible": true,  "opacity": 1.0 },
        "electrical.power":      { "visible": true,  "color-override": "#FF6600" },
        "electrical.low_voltage":{ "visible": true,  "color-override": "#0066FF" },
        "furniture":             { "visible": false }
      },
      "print": true,
      "scale": "1:50"
    },
    "furniture-plan": {
      "label": "Furniture Plan",
      "layers": {
        "architecture": { "visible": true, "opacity": 1.0 },
        "furniture":    { "visible": true, "opacity": 1.0 },
        "electrical":   { "visible": false },
        "plumbing":     { "visible": false }
      }
    },
    "print-bw": {
      "label": "Black and White Print",
      "layers": {
        "*": { "color-override": "monochrome" }
      },
      "style-rules": "none",
      "print": true,
      "exclude-layers": ["guides", "dimensions.internal"]
    }
  }
}
```

The `*` wildcard applies monochrome to all layers. `style-rules: none` disables color-based style rules for that view.

Querying layer properties

Formulas and scripts can query custom layer properties:

```
Self.Prop.contractor = Layer("electrical").props.contractor
```

Views can filter by layer properties:

```
{ "condition": "layer.props.trade == 'Electrical'", "visible": true }
```

What Siqiy adds beyond Visio

- Hierarchical layers
- Multiple layer membership per shape
- Custom layer properties
- Full saved views
- Per-view color override
- Formula-level layer access
- Per-view style-rule toggling

This document is a living working document. It evolves with the project.